

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ СИСТЕМ
УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ (ТУСУР)
Кафедра экономической математики, информатики и статистики (ЭМИС)

ПЕРЕГРУЗКА МЕТОДОВ И ОПЕРАТОРОВ

Отчёт по лабораторной работе по дисциплине «Объектно-ориентированное
программирование»

Студент гр.594-2

А. А. А. А. Танцеров

“ 19 ” Октября 2025 г.

Руководитель канд. техн. наук, доцент
каф. ЭМИС ТУСУР

Я. В. Я. В. Костелей

“ ” 2025 г.

Томск 2025

Лабораторная работа №3

«Перегрузка методов и операторов»

Цель работы: изучение механизмов объектно-ориентированного программирования в C++, включая перегрузку операторов и методов.

Идея перегрузки функций заключается в том, что для одного имени можно определить множество функций, которые отличаются друг от друга списком формальных параметров (сигнатурой).

Приведем еще один пример перегрузки функций. Например, нам необходимо создать функцию, которая бы выводила на консоль переменные или константы следующих типов: `int`, `double`, `char*` и `enum` (перечисление). Правила вывода: для `int` – как минимум 10 знаков, для `double` – 10 знаков до запятой и 3 знака после, для `enum` – вывод наименования его элемента в кавычках, для строки – вывод не более 10 символов в кавычках. Можно предположить, что нам нужно определить четыре функции с одним именем, но в качестве входного параметра должны поступать значения разных типов.

Вывод: изучены принципы объектно-ориентированного программирования на языке C++ через разработку класса для управления данными.

Из рассмотренного кода можем понять, что механизм перегрузки функций не только позволяет создавать функции для разных типов формальных параметров, но и определяет механизм, по которому для заданного типа фактических параметров происходит вызов функции, у которой формальные параметры совпадают по типу или же могут быть приводимы к их типу. Такое свойство определения конкретной функции в зависимости от типа обрабатываемых данных вносит свой вклад в понятие полиморфизма времени компиляции. Стоит отметить, что для перегрузки функций необходимо, чтобы одна функция отличалась от другой количеством формальных параметров или сочетанием их типов (сигнатурой), но при этом функции с одинаковыми наименованиями и списком формальных параметров,

но разными возвращаемыми значениями перегрузке не подлежат. Поэтому при следующем варианте реализации функций возникнет ошибка. Для проверки обернем предыдущий код в класс «OurConsoleOutput», а в качестве возвращаемого значения для каждой функции обозначим ссылку на данный класс «OurConsoleOutput&» и вернем разыменованный указатель на текущий экземпляр класса `this` (для того, чтобы передать по ссылке, а не по указателю). Изменим способ вызова функций, как в подразделе «Вывод на консоль». Как будет происходить данный механизм: разыменовав `this`, мы получим ссылку на значение текущего экземпляра класса, далее компилятор определит тип возвращаемого значения как ссылку, и вернет в место вызова данную ссылку. Такой способ позволяет цепочкой «нанизывать» вызовы функций для одного класса.

Задание) Необходимо перегрузить методы и операторы. Выполнить перегрузку конструктора с параметрами: копируемый объект и флаг того, нужно ли из него копировать план или нет. То есть можно в текущий день из предыдущего дня копировать наименование лекарства, план приема и номер дня, увеличенный на единицу, план приема. Выполнить перегрузку оператора «<<» для вывода на консоль: `ostream& operator << (ostream &out, ClassName& ourObject)`. Перегрузите оператор «[]» для установки приема лекарства в заданное время суток. Перегрузить метод проверки необходимости принять лекарство в заданное время суток без параметров – то есть, если параметр не будет определен, будет выводиться результат метода проверки того, что фактический прием за день соответствует плану.

На первом этапе был расширен класс `Medicine` путём перегрузки ключевых операторов и методов. Реализован дополнительный конструктор копирования с флагом копирования плана приёма, позволяющий создавать объекты для следующего дня на основе данных предыдущего с автоматическим увеличением номера дня. Перегружен оператор << для удобного вывода всей информации о приёме лекарства в консоль, а также

оператор [] для интуитивной установки факта приёма в заданное время суток через индексацию.

На втором этапе выполнена перегрузка метода проверки необходимости приёма лекарства: метод needToTake() теперь поддерживает два варианта вызова — с параметром времени суток для проверки конкретного времени и без параметров для проверки соответствия фактического приёма плану за весь день. Все перегруженные операторы и методы протестированы с учётом ранее реализованной системы исключений для обработки некорректных данных, таких как недопустимые индексы времени суток или нарушения логики работы с динамической памятью.

Реализации операторов представлены на листинге 1.

```
Medicine::Medicine(const Medicine& info, bool copy) {
    size_t medicines_name = strlen(info.name);
    name = new char[medicines_name + 1];
    strcpy_s(name, medicines_name + 1, info.name);

    day = info.day + 1;

    if (copy) {
        for (int i = 0; i < requiredReception; i++) {
            planReception[i] = info.planReception[i];
            didIt[i] = false;
        }
    }
    else {
        for (int i = 0; i < requiredReception; i++) {
            planReception[i] = false;
            didIt[i] = false;
        }
    }
}

ostream& operator << (ostream& out, const Medicine& ourObject) {
    out << "Название: " << ourObject.getName()
        << " День: " << ourObject.getDay() << " ";
    return out;
}

bool& Medicine::operator[] (int time) {
    int index = static_cast<int>(time);
    return planReception[index];
}

bool Medicine::needToTake() {
    for (int i = 0; i < requiredReception; i++) {
        if (planReception[i] && !didIt[i] || !planReception[i] &&
            didIt[i]) {
            return false;
        }
    }
    return true;
}
```

Листинг 1 – Перегруженные методы и операторы

Вывод: изучены механизмы объектно-ориентированного программирования в C++, включая перегрузку операторов и методов.